

- A *record* is denoted by the constructor $\text{Record}(a)$, where $a \in A = \mathbb{N} \cup \{\cdot\}$ is an *address*, either a natural number or the distinguished value $\cdot$ which represents an invalid address, similar to a null pointer in other languages. For a record, the address a in question always points to a map $m \in X \rightarrow A$ from field names to addresses of field values.
- A *function* is denoted by the constructor $\text{Function}(a_f)$, where a_f is the address of a map m of the form $m = \{\text{context} : a, \text{formal-argument} : x, \text{body} : s\}$. Here, a is the address of the function's *stack frame* (to be defined in Section 2), x is the function's formal argument, and s is the statement corresponding to the function's body.
- A *null* value is denoted by the singleton value None .

These explicit constructors do not appear literally in the syntax of MITScript programs, which instead consists of expressions and statements. For example, rather than an instance of $\text{Record}(a)$, a program would contain an instance of the expression $\{x_1 : e_1, \dots, x_n : e_n\}$. Instead, values represent the physical objects stored in memory as the program executes, including their under-the-hood bookkeeping.

Program State. The state of an MITScript program is represented by a *heap* h and a *stack* γ . The heap h is a mapping from addresses a to objects allocated on the heap, each of which is either a value v or a *stack frame* σ . The stack γ is a sequence of addresses of stack frames allocated on the heap.

Stack Frames. A *stack frame* $\sigma \in (X \cup \{\text{parent}, \text{global}\}) \rightarrow A$ is a mapping from program variables to addresses that point to the contents of the program variables stored in the heap. A stack frame has two additional distinguished fields. The **parent** field stores the address of the stack frame's parent frame. The **global** field stores a pair (X_g, a) , where X_g is a set of program variables that have been declared as global within the stack frame by **global** statements and a is the address of a distinguished global stack frame.

We use the notation $\gamma = \gamma'; a$ to denote that the stack γ is equal to another stack γ' with an address a added at the top. Similarly, we use the notation $\gamma = a_1; \gamma'; a_2$ to denote that γ is a stack that has a_1 at the bottom, followed by another stack γ' followed by an address a_2 at the top.

Program Execution. The program starts its execution with a distinguished global stack frame $\sigma_g = \{\text{parent} : \cdot, \text{global} : (\{\}, \cdot)\}$. The initial heap h_0 contains only this initial global stack frame and a pre-allocated value for **None**, i.e. $h_0 = \{a_g : \sigma_g, a_{\text{None}} : \text{None}\}$. The initial stack γ_0 contains the address of the global stack frame, i.e. $\gamma_0 = a_g$. Additionally, it should contain any definitions that are needed to appropriately support MITScript's native functions (Section 10).

Statement Evaluation. Figures 3, 10 and 11 present the semantics of statements. The evaluation relation for statements $(\gamma, h, s) \rightarrow \eta$ denotes that execution of a statement s from a stack γ and a heap h yields a *heap-value* η . A heap value η is either a heap h or a *return-value* denoted by the constructor $\text{Return}(h, v)$, where h is a heap and v is a value. The purpose of the distinguished return-value is to enable the semantics specification to appropriately handle cases where **return** statements appear arbitrarily within the body of a function.

Errors. When a program performs an illegal operation, execution must stop, and the interpreter must report one of the following exceptions by printing the type of the exception to console via standard output:

- **UninitializedVariableException:** when the program attempts to read from a variable that is not present in any appropriate stack frame.
- **IllegalCastException:** when the program attempts to apply an operation to a value for which it does not apply.
- **IllegalArithmeticException:** when the program attempts to divide by zero.
- **RuntimeException:** other illegal operations, such as passing too many arguments to a function.

The semantics in the following sections indicates exactly when each exception must be reported.

3 Simple Expressions

Figures 1, 2, 8 and 9 present the semantics of expressions. The evaluation relation for expressions $(\gamma, h, e) \rightarrow (h', v)$ denotes that given a stack γ and a heap h , an expression e evaluates to a new heap h' and a value v .

Constants. For each type of constant value, the rule generates a value corresponding to the constant.

INTEGERCONSTANT	BOOLEANCONSTANTTRUE	BOOLEANCONSTANTFALSE
$\overline{(\gamma, h, n) \rightarrow (h, \text{Integer}(n))}$	$\overline{(\gamma, h, \text{true}) \rightarrow (h, \text{Bool}(\text{true}))}$	$\overline{(\gamma, h, \text{false}) \rightarrow (h, \text{Bool}(\text{false}))}$
STRINGCONSTANT	NONECONSTANT	
$\overline{(\gamma, h, st) \rightarrow (h, \text{String}(st))}$	$\overline{(\gamma, h, \text{None}) \rightarrow (h, \text{None})}$	

Figure 1: Semantics of constant expressions.

Logical Operators. Logical operators can only be applied to Boolean values.

Arithmetic Operators. The arithmetic operators $-$, $*$, and $/$ can only be applied to integer values. The interpreter must generate an **IllegalArithmeticException** for division by zero.

You should implement MITScript's integers under the standard semantics of 32-bit two's-complement signed integers. You do not need to report errors for integer overflow or underflow.

$\frac{\text{LOGICALOPERATION} \quad (\gamma, h, e_1) \rightarrow (h', \text{Bool}(b_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Bool}(b_2))}{(\gamma, h, e_1 \text{ lop } e_2) \rightarrow (h'', \text{Bool}(b_1 \text{ lop } b_2))}$	
$\frac{\text{ARITHMETICOPERATION} \quad (\gamma, h, e_1) \rightarrow (h', \text{Integer}(n_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Integer}(n_2))}{(\gamma, h, e_1 \text{ op } e_2) \rightarrow (h'', \text{Integer}(n_1 \text{ op } n_2))}$	$\frac{\text{UNARYMINUS} \quad (\gamma, h, e) \rightarrow (h', \text{Integer}(n))}{(\gamma, h, -e) \rightarrow (h', \text{Integer}(-n))}$
$\frac{\text{UNARYNOT} \quad (\gamma, h, e) \rightarrow (h', \text{Bool}(b))}{(\gamma, h, !e) \rightarrow (h', \text{Bool}(\neg b))}$	$\frac{\text{COMPARISONOPERATION} \quad (\gamma, h, e_1) \rightarrow (h', \text{Integer}(n_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Integer}(n_2))}{(\gamma, h, e_1 \text{ cmp } e_2) \rightarrow (h'', \text{Bool}(n_1 \text{ cmp } n_2))}$
STRINGCONCATENATION	STRINGCONCATENATIONLEFTCAST
$\frac{}{(\gamma, h, e_1 + e_2) \rightarrow \underline{\hspace{2cm}}}$	$\frac{}{(\gamma, h, e_1 + e_2) \rightarrow \underline{\hspace{2cm}}}$
STRINGCONCATENATIONRIGHTCAST	NONEEQUALITY
$\frac{}{(\gamma, h, e_1 + e_2) \rightarrow \underline{\hspace{2cm}}}$	$\frac{(\gamma, h, e_1) \rightarrow (h', \text{None}) \quad (\gamma, h', e_2) \rightarrow (h'', \text{None})}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(\text{true}))}$
PRIMITIVEEQUALITY	
$\frac{}{(\gamma, h, e_1 == e_2) \rightarrow \underline{\hspace{2cm}}}$	
RECORDEQUALITY	
$\frac{(\gamma, h, e_1) \rightarrow (h', \text{Record}(a_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Record}(a_2))}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(a_1 = a_2))}$	
FUNCTIONEQUALITY	
$\frac{(\gamma, h, e_1) \rightarrow (h', \text{Function}(a_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Function}(a_2))}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(a_1 = a_2))}$	
PRIMITIVEEQUALITYMISMATCHED	
$\frac{}{(\gamma, h, e_1 == e_2) \rightarrow \underline{\hspace{2cm}}}$	

Figure 2: Semantics of arithmetic and logical expressions.

The operator `+` on two integers corresponds to standard integer addition. The operator `+` on two strings corresponds to string concatenation. The operator `+` on a string and a value that is not a string (where either can appear on either side of the `+`) causes the non-string value to be cast to a string.

Unary Operators. The unary minus operator `-` can only be applied to an integer value. The unary negation operator `!` can only be applied to a Boolean value.

Comparison Operators. The comparison operators `<`, `>`, `<=`, `>=` are only defined for integers.

Integers, Booleans, and strings are primitive values. Two primitive values of the same type are equal if their values are equal. The value `None` is equal to itself. Equality between two records checks that their addresses are identical. Similarly, equality between two functions checks that their addresses are identical. Equality between two values of different type returns `false`.

Operator Errors. For the above operators, the interpreter must generate an `IllegalCastException` for any other operation that is not given an explicit semantics by the rules.

4 Variable Scope

Scoping. In MITScript, a program variable x may reside in different stack frames depending on whether it is global. To identify the appropriate stack frame for a newly declared variable, we define in Figure 7 a function named `lookup-write` that takes the address a_1 of the current stack frame, the current heap h , and the variable x , and returns the address of the stack frame into which x should be written. This address is either the address a_2 of the global frame if x is global, or the address a_1 of the current frame otherwise.

Similarly, we define in Figure 6 a function named `lookup-read` that determines the address of the stack frame in which an existing variable x resides. This address is either that of the global frame, the current frame, or some parent of the current frame. If the variable has not been defined in any appropriate stack frame, then the interpreter must raise an `UninitializedVariableException`.

Global. This rule is a no-op; its semantics is only incorporated later during function calls.

$\frac{\text{GLOBAL}}{(\gamma, h, \text{global } x) \rightarrow h}$

Figure 3: Semantics of the `global` statement.

ASSIGN	RECORDFIELDASSIGNMENT	RECORDINDEXASSIGNMENT
$\overline{\text{globals}(x = e) = \{\}}$	$\overline{\text{globals}(e_1.x = e_2) = \{\}}$	$\overline{\text{globals}(e_1[e_2] = e_3) = \{\}}$
IF		WHILE
$\overline{\text{globals}(\text{if } e \text{ } s_1 \text{ else } s_2) = \text{globals}(s_1) \cup \text{globals}(s_2)}$		$\overline{\text{globals}(\text{while } e \text{ } s) = \text{globals}(s)}$
SEQUENCE	GLOBAL	RETURN
$\overline{\text{globals}(s_1; s_2) = \text{globals}(s_1) \cup \text{globals}(s_2)}$	$\overline{\text{globals}(\text{global } x) = \{x\}}$	$\overline{\text{globals}(\text{return } e) = \{\}}$

Figure 4: Definition of the **globals** function.

ASSIGN	RECORDFIELDASSIGNMENT	RECORDINDEXASSIGNMENT
$\overline{\text{assigns}(x = e) = \{x\}}$	$\overline{\text{assigns}(e_1.x = e_2) = \{\}}$	$\overline{\text{assigns}(e_1[e_2] = e_3) = \{\}}$
IF		WHILE
$\overline{\text{assigns}(\text{if } e \text{ } s_1 \text{ else } s_2) = \text{assigns}(s_1) \cup \text{assigns}(s_2)}$		$\overline{\text{assigns}(\text{while } e \text{ } s) = \text{assigns}(s)}$
SEQUENCE	GLOBAL	RETURN
$\overline{\text{assigns}(s_1; s_2) = \text{assigns}(s_1) \cup \text{assigns}(s_2)}$	$\overline{\text{assigns}(\text{global } x) = \{\}}$	$\overline{\text{assigns}(\text{return } e) = \{\}}$

Figure 5: Definition of the **assigns** function.

GLOBALLOOKUPREAD
$\overline{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \in X_g \quad h(a_2) = \sigma_2 \quad x \in \text{dom}(\sigma_2) \quad \sigma_2(x) = a_3}$
$\text{lookup-read}(a_1, h, x) = a_3$
LOCALLOOKUPREAD
$\overline{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g \quad x \in \text{dom}(\sigma_1) \quad \sigma_1(x) = a_3}$
$\text{lookup-read}(a_1, h, x) = a_3$
SCOPEDLOOKUPREAD
$\overline{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g \quad x \notin \text{dom}(\sigma_1) \quad \text{lookup-read}(\sigma(\text{parent}), h, x) = a_3}$
$\text{lookup-read}(a_1, h, x) = a_3$

Figure 6: Semantics of the **lookup-read** function.

$\frac{\text{GLOBALLOOKUPWRITE} \quad h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \in X_g}{\text{lookup-write}(a_1, h, x) = a_2}$	$\frac{\text{LOCALLOOKUPWRITE} \quad h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g}{\text{lookup-write}(a_1, h, x) = a_1}$
--	--

Figure 7: Semantics of the lookup-write function.

5 Data Expressions

Variable Read. This rule specifies the semantics of reading from a variable, using `lookup-read` (Section 4) to obtain the appropriate address of x .

Record Constructor. This rule evaluates all of the expressions inside the list and constructs a `Record` value from fields to values. Note that the initializer expressions must be evaluated in order.

Field Read. These rules evaluate the expression to a `Record` value, from which the field x is looked up. If the record does not contain the named field x , the expression evaluates to `None`. The statement raises an `IllegalCastException` if the result of the evaluation of e is not a record.

Index Read. These rules evaluate the dynamic index value. If it evaluates to a string, the field by that name is obtained from the record. If it evaluates to some other value, then that value is first cast to a string (Section 9). As before, if there is no such field present in the record, then the expression evaluates to `None`. The order of evaluation is that the record is evaluated first, and then the index.

$\frac{\text{VARIABLEREAD} \quad \text{lookup-read}(a_1, h, x) = a_2 \quad h(a_2) = v}{(\gamma; a_1, h, x) \rightarrow (h, v)}$	
$\text{RECORD} \quad \frac{(\gamma, h, e_1) \rightarrow (h_1, v_1) \quad \dots \quad (\gamma, h_{n-1}, e_n) \rightarrow (h_n, v_n) \quad \{a_1, \dots, a_{n+1}\} \notin \text{dom}(h_n) \quad m = \{x_1 : a_1, \dots, x_n : a_n\} \quad h' = h_n[a_1 : v_1, \dots, a_n : v_n, a_{n+1} : m]}{(\gamma, h, \{x_1 : e_1, \dots, x_n : e_n\}) \rightarrow (h', \text{Record}(a_{n+1}))}$	
FIELDREAD	FIELDREADFAIL
$\frac{}{(\gamma, h, e.x) \rightarrow \underline{\hspace{2cm}}}$	$\frac{}{(\gamma, h, e.x) \rightarrow \underline{\hspace{2cm}}}$
INDEXREAD	INDEXREADFAIL
$\frac{}{(\gamma, h, e_1[e_2]) \rightarrow \underline{\hspace{2cm}}}$	$\frac{}{(\gamma, h, e_1[e_2]) \rightarrow \underline{\hspace{2cm}}}$

Figure 8: Semantics of data expressions.

6 Function Call Expressions

Function Creation. This rule creates a new function value that captures the current frame.

Function Call. There are multiple steps to this rule. First, the base expression for the function call is evaluated to a function value, which points to a map containing the address of a stack frame and the code for the function.

The next step is to allocate a new stack frame for the function call. The rule creates a new stack frame whose parent is the function's stack frame. Argument expressions are evaluated in the caller's frame, and each parameter is bound to the corresponding argument value in the new call frame. The rule traverses the body of the function, and for every global declaration `global x`, it adds x to the set of global variables for the new stack frame. Then, for every assignment of the form $x = e$ in the body of f , the rule adds variable x to the stack frame and initializes it to `None`, unless x is a parameter. A helper function `assigns` (Figure 5) performs this traversal. Finally, the rule evaluates the body of the function with the new stack and heap.

When there are multiple arguments, they must be evaluated in order from left to right. If e_1 does not evaluate to a `Function` value, then `IllegalCastException` must be raised. If the caller passes too many or too few arguments to the function, then `RuntimeException` must be raised. [For this deliverable, write the `FUNCTIONCALL` rule for a function with one argument.](#)

Return. These rules specify that a function call evaluates to the return-value if it exists or else to `None`.

$\frac{\text{FUNCTION} \quad m = \{\text{context} : a, \text{formal-argument} : x, \text{body} : s\} \quad a_f \notin \text{dom}(h) \quad h' = h[a_f : m]}{(\gamma; a, h, \text{fun } x \text{ } s) \rightarrow (h', \text{Function}(a_f))}$	
FUNCTIONCALL	
$(a_g; \gamma; a, h_1, e_1(e_2)) \rightarrow \underline{\hspace{2cm}}$	
$\frac{\text{FUNCTIONCALLRETURN} \quad (\gamma, h, e_1(e_2)) \rightarrow \text{Return}(h', v)}{(\gamma, h, e_1(e_2)) \rightarrow (h', v)}$	$\frac{\text{FUNCTIONCALLNORETURN} \quad (\gamma, h, e_1(e_2)) \rightarrow h'}{(\gamma, h, e_1(e_2)) \rightarrow (h', \text{None})}$

Figure 9: Semantics of function creation and call expressions.

7 Data Statements

Variable Assignment. This rule first evaluates the expression e to a value v and a new heap h' . First, it uses **lookup-write** (Section 4) to determine the address a_2 of the stack frame σ in which to update the variable x , using the appropriate scoping rules. It allocates an new address a_3 previously unused in h' . It returns a new stack with x set to a_3 and a new heap with a_3 pointing to v and a_2 pointing to the new stack.

Record Field Assignment. This rule does not not update a stack frame but rather updates a record in the heap. This statement must raise an **IllegalCastException** if the result of the evaluation of e_1 is not a record.

Record Index Assignment. This rule is similar to the previous one, except the field name is dynamic. The index expression is evaluated and cast to a string using the **str** function, which is defined in Section 9. The operation must raise an **IllegalCastException** if the result of evaluation of e_1 is not a record.

VARASSIGNMENT $\frac{(\gamma; a_1, h, e) \rightarrow (h', v) \quad \text{lookup-write}(a_1, h', x) = a_2 \quad \sigma = h'(a_2) \quad a_3 \notin \text{dom}(h') \quad \sigma' = \sigma[x : a_3]}{(\gamma; a_1, h, x = e) \rightarrow h'[a_3 : v][a_2 : \sigma']}$	
$\text{RECORDFIELDASSIGNMENT}$ $\frac{}{(\gamma, h, e_1.x = e_2) \rightarrow \underline{\hspace{2cm}}}$	$\text{RECORDINDEXASSIGNMENT}$ $\frac{}{(\gamma, h, e_1[e_2] = e_3) \rightarrow \underline{\hspace{2cm}}}$

Figure 10: Semantics of data statements.

8 Control Statements

If. These rules execute the first branch if the condition evaluates to true and the second branch otherwise. The statement must raise an `IllegalCastException` if the result of the evaluation of e is not a Boolean.

While. This rule repeatedly executes its body while the condition evaluates to true. The statement must raise an `IllegalCastException` if the result of the evaluation of e is not a Boolean.

Sequence. These rules evaluate the first statement and then evaluate the second statement. [We have supplied two rules as a hint that the behavior can be usefully divided into two cases.](#)

Return. A `return` statement evaluates its expression, yields a return-value, and terminates execution of the current function.

IFTRUE $\frac{(\gamma, h, e) \rightarrow (h', \text{Bool}(\text{true})) \quad (\gamma, h', s_1) \rightarrow \eta}{(\gamma, h, \text{if } e \text{ } s_1 \text{ else } s_2) \rightarrow \eta}$	IFFALSE $\frac{(\gamma, h, e) \rightarrow (h', \text{Bool}(\text{false})) \quad (\gamma, h', s_2) \rightarrow \eta}{(\gamma, h, \text{if } e \text{ } s_1 \text{ else } s_2) \rightarrow \eta}$
WHILE $\frac{(\gamma, h, \text{if } e \text{ } (s; \text{while } e \text{ } s)) \rightarrow \eta}{(\gamma, h, \text{while } e \text{ } s) \rightarrow \eta}$	SEQUENCE-1 $\frac{}{(\gamma, h, s_1; s_2) \rightarrow \underline{\hspace{2cm}}}$
SEQUENCE-2 $\frac{}{(\gamma, h, s_1; s_2) \rightarrow \underline{\hspace{2cm}}}$	RETURN $\frac{(\gamma, h, e) \rightarrow (h', v)}{(\gamma, h, \text{return } e) \rightarrow \text{Return}(h', v)}$

Figure 11: Semantics of control statements.

9 Cast Semantics

The semantics of MITScript requires that certain values be automatically converted into strings. This conversion is performed via a function called `str`, defined by the following rules:

- `str(String(st)) = st`
- `str(Boolean(b)) = “true”` if *b* is true, or “false” otherwise
- `str(Integer(n)) =` base-10 representation of *n*, with no leading zeroes, preceded by sign if negative
- `str(Function(af)) = “FUNCTION”`
- `str({xi : vi, ...}) = “{” + xi + “:” + str(vi) + “ ” + ... + “}”`
 - Fields should be ordered lexicographically in the resulting string (i.e., $i < j \implies x_i < x_j$)
- `str(None) = “None”`

10 Native Functions

In addition to the core syntax and semantics of MITScript, your interpreter must support the following three native functions:

- `print(s)` Cast *s* to a string and print it to the console (via standard output) followed by a newline.
- `input()` Read a line of input from the console (via standard input) and return it as a string value.
- `intcast(s)` Parse *s* as an integer value. If the string does not represent an integer (e.g., “hello”), this function raises an `IllegalCastException`. You may internally utilize the C function `atoi`.

These functions should be initially available in the global scope. However, it should be possible for the program to redefine them. For example, the program below updates `print` to add a prefix to every message:

```
print("Hello"); // this should print 'Hello' to the console.
oldprint = print;
print = fun(s){
    oldprint("OUTPUT: " + s);
};
print("Hello"); // this should print 'OUTPUT: Hello' to the console.
```

You should also implement equality for native functions. Comparing a native function with f_1 with f_2 should only return true if f_1 and f_2 both refer to the same native function. Example behavior is given below.

```
print(input == input); // this should print 'true' to the console.  
print(input == intcast); // this should print 'false' to the console.  
also_input = input;  
print(also_input == input); // this should print 'true' to the console.
```